

Project 01 Performance Measurements of Crux

Due: Feb 18, 2026

This project evaluates the computational and communicative performance of the **Crux cluster**. It contains the following two parts:

Part I: MPI π Scaling Study on Crux

Part II: FLOPs Measurement

Preparation

Before starting, please complete the hands-on PBS job-submission tutorial to get yourself familiar with how to run jobs on supercomputer.

https://github.com/zhenghh04/csci394-spring26/blob/main/00_pbs/README.md

Part 1. MPI π Scaling Study on Crux

Objective: Analyze the parallel performance of the Crux cluster by calculating the value of π using the Monte Carlo method.

Run **strong** and **weak** scaling tests for the MPI pi program on Crux using **32 MPI ranks per node** and scaling up to **128 nodes**. You will collect runtime data and compute speedup/efficiency for strong scaling.

Program: 04_benchmarks/scaling/pi_mpi4py.py

[git@github.com:zhenghh04/csci394-spring26.git](https://github.com/zhenghh04/csci394-spring26.git)

Required scale points

Use these node counts: 1, 2, 4, 8, 16, 32, 64, 128

With 32 ranks per node, total ranks are: 32, 64, 128, 256, 512, 1024, 2048, 4096

Strong scaling (fixed global work)

Choose a single global sample count (e.g., `SAMPLES=1_000_000_000`) and keep it constant for all node counts.

Command pattern:

```
mpirun -n <ranks> --ppn 32 --cpu-bind depth -d 1 python pi_mpi4py.py --samples <SAMPLES>
```

Weak scaling (fixed work per rank)

Choose a per-rank sample count (e.g., `SAMPLES_PER_RANK=100_000_000`) and compute `global_samples = samples_per_rank * total_ranks` for each run.

Command pattern:

```
total=$((SAMPLES_PER_RANK * ranks))
```

```
mpirun -n <ranks> --ppn 32 --cpu-bind depth -d 1 python pi_mpi4py.py --samples $total
```

PBS batch template (Crux)

```
#!/bin/bash
#PBS -A DLIO
#PBS -l select=2
#PBS -l walltime=00:30:00
#PBS -N mpi_pi_scaling
#PBS -l filesystems=eagle:home
#PBS -j oe
```

```
cd ${PBS_O_WORKDIR}
source /eagle/datasets/soft/crux/miniconda3.sh
export NUM_NODES=$(cat $PBS_NODEFILE | uniq | wc -l)
```

```
# Strong scaling (fixed global work)
SAMPLES=1000000000
mpirun -n $RANKS --ppn $PPN --cpu-bind depth -d 1 \
python pi_mpi4py.py --samples $SAMPLES
```

```
SAMPLES_PER_RANK=100000000
mpirun -n $RANKS --ppn $PPN --cpu-bind depth -d 1 \
python pi_mpi4py.py --samples $((SAMPLES_PER_RANK * RANKS))
```

You can do `qsub -l select=N qsub.sc` and vary `N` to run on different nodes with the same submission script, where `N` is the number of nodes to run the jobs. “`-l select=N`” will override the `#PBS -l select=2` inside the submission script.

Part II. FLOPs Measurement

Objective: Determine the practical peak performance of the Crux cluster using the industry-standard **High-Performance Linpack (HPL)**.

- **Algorithm:** HPL solves a dense $N \times N$ system of linear equations ($Ax=b$) using LU factorization with row partial pivoting.
- **Configuration (HPL.dat):** Requires tuning parameters like the problem size (N), block size (NB), and process grid dimensions ($P \times Q$).
- **Calculation:** Performance is reported in **GFLOPS** or **TFLOPS**, representing the actual billions/trillions of operations completed per second.

Please follow the instructions to measure the FLOPs on 1, 4, 16, 64, 128 nodes:

https://github.com/zhenghh04/csci394-spring26/blob/main/04_benchmarks/hpl/README.md

Please compare the theoretical flops and compute the efficiency. (Measured / theoretical)

The theoretical peak performance can be computed based on the following information:

- Each node on **Crux** has **2× AMD EPYC 7742 (Rome)** processors.
- Each EPYC 7742 has **64 cores**.
- Each core supports **AVX2 (256-bit) with FMA**, enabling **16 FP64 FLOPs per cycle per core**.

How to submit your results

Please submit a **word / PDF document** with the following information

For Part 1.

1. Quote of qsub.sc.
2. Two tables with columns one for strong scaling and one for weak scaling:
 - nodes, samples, time_s, scaling efficiency
3. Two plots (one for each scaling type): Time vs num-nodes
 - Strong: time_s vs nodes
 - Weak: time_s vs nodes

For Part II.

1. Output of your runs on different node counts.
2. A table show the measured flops and theoretical flops and efficiency